

Guida allo sviluppo e al rilascio

Riferimento di rilascio: il profilo XLSX statico è incluso nella wheel qualificata `f426c5aa...`; non appartiene alla precedente `039e0fd8...`. Le suite installate su Python 3.13 e 3.11 sono concluse, così come installazione pulita e upgrade. Per hash ed esiti esatti consultare [VALIDAZIONE.md](#). Ogni successiva modifica richiede un'attribuzione distinta.

Questa guida riguarda il repository di MCP Integrity Guard 1.0.0. Per installazione, operazioni, persistenza, API e recovery leggere [MANUALE_TECNICO.md](#); per contratti dettagliati usare [CONTRACT.md](#) e i documenti specialistici collegati in fondo.

1. Struttura del repository

Percorso	Contenuto
<code>backend/integrity_guard/</code>	Pacchetto Python e regole confezionate
<code>backend/tests/</code>	Test unitari e integrazioni, compresi subprocessi Linux reali
<code>backend/pyproject.toml</code>	Metadata, intervalli delle dipendenze e entry point <code>mcp-guard</code>
<code>backend/requirements-linux.lock</code>	Versioni esatte delle dipendenze del runtime Linux validato
<code>frontend/src/</code>	Applicazione React/TypeScript
<code>frontend/package-lock.json</code>	Albero delle dipendenze frontend risolte
<code>frontend/tests/</code>	Verifiche browser del layout e dei flussi pertinenti
<code>deploy/linux/</code>	Installer, launcher, wrapper CLI/MCP e verificatore release
<code>examples/</code>	Documenti e configurazioni dimostrativi, senza server da avviare
<code>qa/</code>	Harness di durata, launcher separati, verifiche e benchmark
<code>docs/</code>	Documentazione di prodotto, contratti specialistici e validazione
<code>package_app.py</code>	Costruzione del distributable Linux da wheel e dist già costruiti

`.venv`, runtime, `frontend/dist`, `node_modules`, `.test-data`, dati di sviluppo, log e cache sono esclusi da Git. Non usare questa esclusione come unica protezione: prima di un commit controllare i file selezionati. Non versionare chiavi, token, database, backup, documenti degli utenti o screenshot contenenti credenziali.

Le regole del repository sono in [AGENTS.md](#). In particolare, i file scansionati e i comandi MCP scoperti non vengono eseguiti; non si aggiunge telemetria esterna.

2. Preparare l'ambiente

Prerequisiti: Python almeno 3.11 con `venv`, toolchain Node/npm compatibile con le dipendenze frontend, e un host Linux con Landlock/libseccomp per provare realmente i worker. Vite 6 dichiara Node `^18.0.0 || ^20.0.0 || >=22.0.0`; scegliere una versione supportata dalla propria politica di manutenzione e registrare quella usata. Questo intervallo del package non attesta la manutenzione di sicurezza di ogni versione Node.

Dalla root del repository:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r backend/requirements-linux.lock
.venv/bin/python -m pip install -e './backend[dev]'
cd frontend
npm ci
cd ..
```

Il lock Linux contiene versioni esatte, mentre `pyproject.toml` definisce gli intervalli compatibili e gli strumenti di sviluppo. `json5` è fissato a 0.15.0 perché il controllo del budget usa interfacce del parser che devono essere verificate prima di aggiornarlo. Non rigenerare il lock incidentalmente durante una modifica alla UI.

La serie 1.0.0 usa React 19, Vite 6, TypeScript 5 e FastAPI/Pydantic 2; le versioni puntuali effettive si ricavano dai lock e dall'ambiente installato. La compatibilità Python non si deduce dal solo `requires-python`: le prove su ciascun interprete restano necessarie. Consultare [VALIDAZIONE.md](#) per ciò che è stato realmente eseguito.

Avvio dello sviluppo:

```
bash start-dev.sh
```

Lo script serve Vite su `127.0.0.1:5173` e il backend con reload su `127.0.0.1:8766`, usando `.dev-data-linux` nella root del repository. Configura il proxy `/api` e termina il backend quando si chiude lo script. La riga relativa al token indica il percorso del file, non il suo valore. Non impostare questa directory ai dati di produzione.

In sviluppo `PYTHONPATH` include il sorgente per il reload. I launcher distribuiti e i worker usano invece `-I`, che esclude directory corrente e `PYTHONPATH` dalla risoluzione dei moduli. Non rimuovere questo isolamento per aggirare un problema di import; controllare quale pacchetto è installato nel venv selezionato.

3. Come orientare una modifica

Modifica	Punti principali e verifiche pertinenti
Configurazione/hash/diff	<code>canonical.py</code> , <code>core.py</code> ; fixture di parsing, normalizzazione, versioni e approvazioni
Gate/firma/policy/audit	<code>core.py</code> , <code>test_core.py</code> , <code>test_audit_cache.py</code> , <code>test_core_limits.py</code>
Inventario visualizzato	Proiezione in <code>core.py</code> , <code>monitor</code> , tipi e componenti React; <code>test_snapshot_projection.py</code> e API
Formato o estrazione	<code>extraction.py</code> , <code>xlsx_extractor.py</code> , registro estrattori e worker; test del formato e dei budget
Regola scanner	<code>rules.json</code> , <code>scanner.py</code> ; casi positivi, negativi e limiti già previsti
Supervisione/isolation	<code>worker_process.py</code> , <code>scan_worker.py</code> , <code>linux_sandbox.py</code> ; test su subprocessi reali
Copia HTML	Moduli <code>html_text</code> , <code>sanitize_*</code> , <code>sanitizer</code> ; protocollo e doppia scansione

Modifica	Punti principali e verifiche pertinenti
CLI/client/MCP	<code>cli.py</code> , <code>connection.py</code> , <code>mcp_server.py</code> ; trasporto e byte effettivamente consegnati
Monitor cartelle	<code>filewatch.py</code> ; continuità delle visite, cache, percorsi e report persistiti
UI	<code>App.tsx</code> , <code>Components.tsx</code> , moduli di scansione/operazioni e tipi condivisi; build e flussi browser

Un cambiamento di presentazione non deve trasformare lo stato dell'inventario in un'autorizzazione. Mantenere separati `snapshot_valid`, validità della sorgente, approvazione firmata e decisione corrente del gate. Dopo refresh, modifica o azione, la UI deve riferire hash e versione del componente realmente mostrato, gestire conflitti e distinguere errori da esiti negati.

Per nuove interfacce partire dal contratto di input/output. La serializzazione deve mantenere scalari e tipi previsti, limiti, identificativi, hash e motivi di fallimento; non far passare testo sorgente attraverso errori del trasporto. Aggiornare documentazione, tipi frontend e test del comportamento osservabile insieme al codice.

Invarianti da preservare

- Un'approvazione si riferisce alla precisa versione e policy: nessun aggiornamento può trasferire fiducia in base al nome del componente.
- RAW, canonico e fingerprint coprono i contenuti originali, compresi i campi mascherati. Un ritorno a byte precedenti non ripristina automaticamente fiducia.
- Il gate rilegge la sorgente; il polling dell'inventario verifica snapshot persistiti senza fare una nuova discovery o un gate per ogni riga.
- Un output worker deve avere schema, hash, dimensione e stato coerenti. Timeout, analisi parziale e ambiguità negano l'uso.
- Il confinamento viene applicato prima di input e parser dei documenti; il parent non trasferisce segreti al worker.
- `SANITIZED` non sostituisce `VALID / ALLOWED`; la consegna richiede una nuova scansione dei byte derivati e una registrazione riuscita.
- Database, chiave e checkpoint formano un insieme da preservare. Non "riparare" automaticamente l'audit rimuovendo evidenze.

Non alterare il profilo canonico, le firme o il significato della policy senza un piano esplicito di compatibilità e riapprovazione. Un formato del record persistito non è soltanto un tipo TypeScript: influenza storico, firme, cache e recovery.

4. Regole, parser ed estensioni

Le regole di produzione sono confezionate in `rules.json`. Ogni regola ha ID, titolo, severità, categoria, punteggio e pattern per lingua. La versione del catalogo è riportata nei report. Il costruttore compila e valida il catalogo; limiti e prefiltri non devono cambiare il risultato delle regole applicabili.

Non esistono un aggiornamento remoto automatico delle regole o un endpoint per sostituirle. La UI di policy modifica azioni e soglie, non il catalogo. Il parametro Python `Scanner(rules_path=...)` è una superficie per codice fidato; l'API ordinaria usa il catalogo del pacchetto installato.

Per aggiornare una regola:

1. Identificare la condizione osservabile e il comportamento desiderato, compresi esempi leciti da non segnalare.
2. Modificare il catalogo e la sua versione; mantenere ID stabili quando il significato è lo stesso.
3. Verificare i casi pertinenti, il budget e il funzionamento attraverso il worker confinato, oltre alla chiamata Python diretta.
4. Ricostruire e verificare la wheel, installarla in una copia separata e riavviare il servizio dopo la distribuzione controllata.

La cache del monitor include hash delle regole, codice, parser, policy, registro, Python e Unicode. L'invalidazione richiede nuove scansioni; non modifica automaticamente i report storici. Non cambiare file nel runtime mentre i processi li stanno usando: distribuirne una copia coerente e distinta.

`canonical.register_adapter` registra parser di configurazione tramite codice applicativo fidato. L'estensibilità dei documenti è separata: [ESTRATTORI.md](#) descrive `register_extractor`, bootstrap statico, congelamento e fingerprint dei file confezionati/dependenze. Non sono previsti entry point individuati automaticamente, import da documenti, directory plugin degli utenti o override dei formati incorporati. Nella release 039e il registro aggiuntivo era vuoto. La wheel `f426c5aa...` registra esplicitamente il solo profilo XLSX statico, handler `xlsx_extractor.extract_xlsx`, versione `1.0.0`, dipendenza dichiarata `defusedxml` già fissata a `0.7.1` nel lock. PPTX, EML, MSG, RTF e ODT restano non abilitati; DOC/XLS legacy, XLSM e XLSB rimangono non supportati.

La funzione handler riceve byte, nome e `Budget`, restituisce `Extraction` e usa i controlli comuni di completamento/accounting. Il bootstrap deve essere riproducibile in un worker nuovo prima dell'input. I limiti del manifest del registro sono 256 file Python/JSON e 4 MiB, con fonti regolari e stabili. Un helper confezionato cambiato deve modificare il fingerprint anche se il nome dell'handler resta uguale.

Sviluppare sul profilo XLSX

Il profilo corrente legge il sottoinsieme Transitional di celle e metadati statici: non deve essere presentato come interprete Excel. Formule di qualsiasi tipo, nomi definiti, media, oggetti attivi, dati esterni e strutture non ispezionate interrompono l'analisi. I normali hyperlink vengono trattati come metadati e non visitati. Nessun ricalcolo, OCR, conversione dei formati legacy o estrazione del solo cached value può essere implicito. Il [manuale tecnico](#) elenca limiti, livelli e codici.

Le verifiche pertinenti sono `backend/tests/test_xlsx_extractor.py`, `test_extractor_registry.py`, `test_extractor_integration.py` e i casi di classificazione in `test_scanner.py`. Usare quelle fixture ordinarie già presenti; includere le prove tramite worker reale oltre alle chiamate dirette. Un file non ZIP nominato `.xlsx` è ora malformato perché l'estensione è registrata; `.xls` resta un esempio di formato non supportato. Questa differenza non va trattata come un indebolimento del blocco.

Ogni estensione del profilo richiede copertura dei nuovi contenuti, contabilità delle viste e dei riferimenti ripetuti, limiti ZIP/XML e risultato completo coerente. Aggiornare il codice fidato e il fingerprint del pacchetto, costruire un candidato distinto e attribuire gli esiti alla wheel e all'interprete effettivamente provati. I risultati mirati sul sorgente non attestano da soli l'installazione o la compatibilità di rilascio su Python 3.11.

5. Verificare il cambiamento

Eseguire i test pertinenti alla modifica, poi i controlli di rilascio richiesti. Non dichiarare superato un controllo saltato perché il sistema non dispone del sandbox. Le prove del confinamento devono essere eseguite su almeno un host compatibile per una release Linux.

```
.venv/bin/python -m pytest backend/tests -q
.venv/bin/python -m unittest discover -s qa -p 'test_*.py'
cd frontend
npm run format:check
npm run build
cd ..
```

I test backend comprendono configurazioni, approvazioni, persistenza, API, protocolli, scanner, trasformazioni e processi reali. Le prove QA dell'harness non sono ulteriori prove dello stesso prodotto: attribuirle separatamente. Per modifiche limitate usare inizialmente i moduli pertinenti; ripetere suite complete quando cambia il candidato o il piano di rilascio lo richiede.

La suite esistente è un insieme di casi e regressioni, non un inventario di tutti i file malevoli possibili. Un numero elevato di campioni non dimostra copertura universale; distinguere formati, condizioni osservate e comportamenti effettivamente verificati. Un falso positivo segnala come problematico un contenuto lecito, un falso negativo non rileva un problema presente. Nessun insieme finito di file dimostra l'assenza di entrambi. Per i casi già disponibili fare riferimento a `backend/tests/` e ai risultati attribuiti in [VALIDAZIONE.md](#), mantenendo espliciti limiti e campioni. Questa guida non introduce generatori di payload o nuovi corpus offensivi.

Le verifiche browser richiedono un backend QA dedicato, dati sintetici e Playwright con browser disponibile. Lo script `mobile-layout.cjs` richiede almeno una scansione salvata e accetta un **percorso di token QA**, non un token sulla riga di comando. Playwright non è una dipendenza della UI distribuita. Esempio di toolchain QA separata:

```
npm install --prefix .test-data/browser-tools playwright
.test-data/browser-tools/node_modules/.bin/playwright install chromium
node frontend/tests/mobile-layout.cjs \
  "$CHEKER_QA_DATA/api-token" \
  "$(pwd)/.test-data/browser-tools/node_modules/playwright" \
  "http://127.0.0.1:8768" \
  "$(pwd)/.test-data/browser-shots"
```

`CHEKER_QA_DATA` deve indicare la directory del backend QA già predisposto, con porta 8768 nell'esempio. Le dipendenze di sistema del browser sono separate da quelle del programma. Non puntare la QA ai dati dell'utente e non includere token, frammenti URL o documenti privati in screenshot e report.

OpenAPI locale senza store o credenziali

`create_app()` normalmente crea token, apre database e prepara i servizi. Per ispezionare soltanto lo schema, usare stub dei servizi e una directory esistente; non entrare nel lifespan e non chiamare rotte. Dalla root del repository, con il venv installato:

```

from contextlib import ExitStack
from pathlib import Path
from unittest.mock import MagicMock, patch
from integrity_guard import api

with ExitStack() as stack:
    for name in ("GuardStore", "Reports", "Sanitizer", "Monitor", "FileWatch"):
        stack.enter_context(patch.object(api, name, MagicMock()))
    stack.enter_context(patch(
        "integrity_guard.linux_sandbox.probe_capabilities",
        return_value={"available": False, "error": "Schema inspection only"},
    ))
    app = api.create_app(
        data_dir=Path.cwd(),
        token="documentation-placeholder-not-a-real-credential",
        start_monitor=False,
    )
    schema = app.openapi()
    print(schema["openapi"], len(schema["paths"]))

```

Questo procedimento è per documentazione, non per collaudare il servizio. Per la versione documentata lo schema contiene 30 percorsi e 34 operazioni API. Swagger/OpenAPI HTTP sono disabilitati; l'autenticazione middleware non viene dichiarata automaticamente nello schema. Le risposte dinamiche non sono tutte modellate in dettaglio: verificare anche [CONTRACT.md](#), codice e test. Gli errori 422 possono avere `detail` strutturato, a differenza dei normali messaggi applicativi testuali.

L'export audit richiede una nota esplicita in qualsiasi client generato: pur passando un valore più alto dal router, lo store restituisce **al massimo 100.000 eventi**. Non inventare un parametro di paginazione assente dallo schema.

Durata, risorse e prestazioni

`soak.py` esercita scansioni, monitor, configurazioni, gate, registro e riavvii. `soak_features.py` aggiunge una copia HTML innocua a ogni ciclo, verifica byte/hash/due report e pagina i metadati. I launcher separati registrano PID e start ticks; richiedono directory nuove sotto un segmento `.test-data` e permessi privati su filesystem Linux nativo.

Esempio di prova breve su **runtime di release già installato**, con radici nuove e distinte:

```

python3 -I qa/launch_soak_features.py \
--python "$CHEKER_RELEASE_DIR/runtime-linux/bin/python" \
--data-root /tmp/cheker-qa/.test-data/run-001 \
--control-dir /tmp/cheker-qa/.test-data/control-001 \
--duration 180 --interval 30 --restart-every 0

```

Impostare `CHEKER_RELEASE_DIR` al candidato installato. Non riutilizzare i nomi dopo una prova, neppure fallita. `--restart-every 0` misura continuità dello stesso backend; un valore positivo prova anche il riavvio. Queste coperture non sono equivalenti. Le risorse vengono campionate ogni secondo; `progress.json` è avanzamento, `report.json` è l'esito finale. Un'interruzione, un controller orfano o un fallimento non diventano PASS. Arrestare soltanto processi di cui corrispondono PID e start ticks e conservare i risultati effettivi.

Il runner HTML vincola l'harness baseline a un hash atteso. Una modifica intenzionale al baseline richiede di aggiornare quel vincolo con le relative prove; non eliminare il controllo per far partire una run. Non cambiare codice o runtime mentre un collaudo di durata li usa.

Per benchmark usare `benchmark_audit.py` e i limiti documentati in [PERFORMANCE.md](#). `benchmark_snapshot_projection.py` riproduce un confronto storico con commit e hash sorgente fissati: non è un benchmark generico per qualsiasi checkout e può richiedere cronologia Git disponibile. Registrare quantità di eventi/componenti/byte, interprete, filesystem, carico concorrente e campioni; non attribuire a una nuova build misure ottenute su un'altra.

6. Build e pacchetto distribuibile

La UI deve essere compilata e la wheel costruita prima di `package_app.py`. Usare **sempre il percorso assoluto** del backend: `pip wheel backend` può riferirsi a un pacchetto omonimo dell'indice, anziché al repository.

```
CHEKER_SOURCE_DIR="$(pwd)"
mkdir -p .test-data
CHEKER_RELEASE_STAGE="$(mktemp -d "$CHEKER_SOURCE_DIR/.test-data/release-XXXXXX")"
mkdir -p "$CHEKER_RELEASE_STAGE/wheels"
.venv/bin/python -m pip wheel --no-deps \
  --wheel-dir "$CHEKER_RELEASE_STAGE/wheels" \
  "$CHEKER_SOURCE_DIR/backend"
cd frontend
npm ci
npm run build
cd ..
.venv/bin/python package_app.py --app-dir "$CHEKER_RELEASE_STAGE"
python3 "$CHEKER_RELEASE_STAGE/verify-release.py"
```

Il packager confronta i file Python/JSON della wheel con quelli sorgente e rifiuta una wheel obsoleta. Non costruisce la wheel e non installa dipendenze. Copia UI, esempi, script, lock e documentazione, genera un manifest SHA-256 e il tar con checksum. Il file originale dei requisiti non viene incluso tra i documenti della distribuzione. L'archivio non contiene dati privati o virtualenv.

Usare il packager soltanto su uno staging nuovo. Il programma ricrea le directory UI ed esempi della destinazione; non è un publisher incrementale per l'app aperta. Il flag Windows opzionale non costituisce una validazione del prodotto per Windows e non deve entrare nella procedura Linux.

Nel tar la root è `0700`, gli script shell `0755`, gli altri file `0644`; i file hanno UID/GID zero. Il verificatore controlla i file elencati dal manifest, non firma la provenienza del distributore. Controllare anche elenco dei membri, assenza di link/percorsi estranei/dati e corrispondenza dei byte. Non costruire archivi ricorsivi prendendo tutta la cartella di un'app già installata.

I documenti Markdown aggiunti a `docs/` vengono confezionati automaticamente, salvo l'esclusione citata. Un publisher con allowlist esterna o confronto rigido dei nomi deve essere aggiornato esplicitamente per aggiungerli; non allargare indiscriminatamente il confronto agli altri file.

Verifica del candidato installato

Installare il tar in una directory Linux pulita e nuova, poi verificare:

- Checksum e manifest prima dell'installazione; wheel e payload attesi dopo l'installazione.
- `runtime-linux/bin/python -m pip check` e diagnostica confinata `READY`.
- Avvio e arresto ordinati con dati QA nuovi, API, CLI e lettore MCP pertinenti.
- UI servita dal pacchetto, comprese approvazione obsoleta, aggiornamento del dettaglio, errori e layout previsti.

- Aggiornamento di una copia popolata: componenti, approvazioni, hash, policy, report e radici conservati, senza perdere chiave o checkpoint.
- Suite e prove di durata previste, attribuite all'hash effettivamente installato.

La sola importazione dal checkout non collauda la wheel. Non risolvere il symlink finale di `runtime-linux/bin/python` verso il Python di sistema: cambierebbe il contesto del venv usato. Una release senza risultati conclusi non deve essere presentata come già collaudata.

Aggiornamento della sola UI

La UI può essere distribuita senza riavviare il backend solo dopo verifica del dist esatto e delle interfacce con la wheel installata. Aggiungere prima i nuovi asset con nome dipendente dal contenuto, sostituire `index.html` atomicamente per ultimo fra i file UI e conservare gli asset precedenti live per i client aperti. Non sovrascrivere con byte diversi un asset che mantiene lo stesso nome.

Il nuovo manifest/tar deve descrivere il nuovo dist verificato, non incorporare gli asset obsoleti conservati soltanto live. Registrare separatamente i loro hash. Verificare i byte live prima di sostituire manifest/tar/checksum e conservare un piano di rollback. Le sostituzioni sono atomiche per file, non una transazione fra tutti i file. Non cambiare wheel, dipendenze, launcher o dati in un aggiornamento dichiarato della sola UI.

7. Versioni, contributi e manutenzione

Le versioni del pacchetto sono dichiarate in `backend/pyproject.toml`, `backend/integrity_guard/__init__.py`, `frontend/package.json` e `package_app.py`. Il catalogo regole e i protocolli hanno versioni proprie. Mantenerle coerenti con il contenuto e con le migrazioni previste; l'etichetta `1.0.0` da sola non identifica un candidato binario.

Per ogni candidato registrare commit o stato sorgente, hash della wheel, file payload, lock, UI dist, manifest e tar. Una modifica alla documentazione cambia manifest/tar senza necessariamente cambiare wheel o UI. Una modifica frontend cambia il dist senza cambiare il backend. Tenere separate queste identità nelle note di rilascio.

Una proposta di modifica deve descrivere problema concreto, comportamento risultante, verifiche effettuate e limiti rimasti. Riportare separatamente PASS, FAIL, SKIP e prove interrotte, senza sommare tentativi sovrapposti per gonfiare un totale. Conservare l'evidenza di un errore dell'harness anche quando una successiva esecuzione corretta passa.

Prima di proporre un commit:

```
git status --short
git diff --check
git diff --stat
```

Esaminare il diff dei file da includere; non aggiungere l'intera area QA ignorata per pubblicare un singolo rapporto. Le issue devono usare fixture sintetiche minime e dettagli di ambiente necessari, senza documenti, token, percorsi personali o dump di database. Per segnalazioni di sicurezza usare il canale privato indicato dal progetto quando disponibile, evitando di pubblicare credenziali o dati reali.

Non sono inclusi un servizio remoto di aggiornamento automatico, un sistema di migrazione/downgrade universale o una pipeline CI implicita. Le verifiche elencate sono passaggi ripetibili da eseguire e registrare; l'aggiunta futura di automazioni deve preservarne confini e attribuzione.

8. Riferimenti

- [MANUALE_TECNICO.md](#): operazioni, API, backup, recovery e limiti.
- [CONTRACT.md](#): payload e interfacce fra moduli.
- [ARCHITETTURA.md](#): descrizione delle scelte e dei confini.
- [LINUX_SANDBOX.md](#): requisiti ed enforcement del worker.
- [MCP_INTEGRATION.md](#): lifecycle, strumenti, radici e trasporto stdio.
- [SANITIZZAZIONE.md](#): profilo HTML e consegna della copia.
- [ESTRATTORI.md](#): registro dei soli estrattori fidati confezionati.
- [REQUISITI_COPERTURA.md](#): corrispondenza con requisiti e differenze dichiarate.
- [VALIDAZIONE.md](#), [PERFORMANCE.md](#): evidenze attribuite e limiti delle misure.